

RESEARCH ARTICLE

A Use-After-Free Vulnerability Detection Method for Multi-Threaded Programs Based on an Improved Petri Net and Value Flow Graph

SHANSHAN LI¹, YUNXIA BAO², FAMING LU¹, (Member, IEEE), CHENG YU³,
AND CONG LIU⁴

¹College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China

²College of Mathematics and System Science, Shandong University of Science and Technology, Qingdao 266590, China

³Qingdao Municipal Bureau of Statistics, Qingdao 266071, China

⁴NOVA Information Management School, Nova University of Lisbon, 1070-312 Lisbon, Portugal

Corresponding author: Yunxia Bao (baoyunxia@sdust.edu.cn)

This work was supported in part by the National Key Research and Development Program of China under Grant 2022ZD0119501, in part by NSFC under Grant 52374221 and Grant 52574256, in part by the Science and Technology Development Fund of Shandong Province of China under Grant ZR2022MF288 and Grant ZR2023MF097, in part by the Taishan Scholar Program of Shandong Province under Grant tstp20250506, and in part by the Natural Science Foundation of Shandong Province of China under Grant ZR2024QF107.

ABSTRACT Use-After-Free (UAF) vulnerability is one of the common vulnerabilities in multi-threaded programs. Its static detection methods based on value flow analysis commonly show good analysis precision and efficiency. However, most of them do not adequately consider the causality constraints caused by different concurrency primitives and complex control structures, which may lead to false positives. Hence, this work proposes a static UAF vulnerability detection method based on an improved Petri net and value flow graph. Firstly, source codes of a multi-threaded program are transformed into intermediate codes, based on which a segmented Petri net is constructed, and control flow causality constraints are analyzed from the net. Then, a static value flow graph of a multi-threaded program is constructed, segment information in the corresponding segmented Petri net is associated with the graph, and UAF triggering and condition-satisfied constraints are analyzed from the graph. Finally, the compatibility among control flow causality, UAF triggering, and condition-satisfied constraints is analyzed to detect UAF vulnerabilities. This work conducts experiments on public and synthetic datasets. Experimental results show that the proposed method's recall rate and precision are 25% and 33.3% higher than those of Canary, respectively; the proposed method's recall rate and precision are 3% and 15.1% higher than those of Saber, respectively; and the proposed method's time complexity is reduced by 12.6 ms and 21.3 ms compared with Canary and Saber, respectively.

INDEX TERMS Multi-threaded program, UAF vulnerability detection, Petri net, value flow graph, segmentation.

I. INTRODUCTION

Multi-threaded programs are a cornerstone technology for industry applications to overcome single-core performance limitations, because they significantly enhance computational efficiency and resource utilization by executing multiple tasks in parallel. Hence, they play important roles in concurrency and latency-sensitive scenes, e.g., industry applications, operating systems, game engines,

and artificial intelligence computations. Although multi-threaded programs improve program performance, they suffer from several potential risks in task decomposition and asynchronous processing, e.g., Use-After-Free (UAF) [1], deadlocks [2], [3], data races [4], [5], and memory leaks [6], which severely impact their security, stability, and reliability. For example, Google updated its Chrome browser (version 88.0.4324.150) in 2021 to fix a UAF vulnerability CVE-2021-21148 [7] in a rendering engine, because it enables attackers to control affected systems through remote code execution in special web pages.

The associate editor coordinating the review of this manuscript and approving it for publication was Zhiwu Li¹.

Existing UAF vulnerability detection methods can be divided into dynamic detection methods and static ones. The former collects a multi-threaded program's running trace and analyzes UAF vulnerabilities in it. The latter analyzes source codes of a multi-threaded program directly and does not require executing it. Their advantages and disadvantages are as follows:

1) For dynamic detection methods, they can accurately detect UAF vulnerabilities in a multi-threaded program's running trace because it contains actual running information. However, a multi-threaded program may have subtle behavior, which needs to be run multiple times. Hence, it is difficult to detect all the whole UAF vulnerabilities in a large multi-threaded program;

2) For static detection methods, they can detect more UAF vulnerabilities than dynamic ones because they directly analyze code logic structures and mine possible execution paths of a multi-threaded program by using segment graphs, value flow graphs, and Petri nets [8], [9]. However, if concurrency primitives and control structures of a multi-threaded program are complex, e.g., there are different loops, conditionals, and lock operations, existing static detection methods cannot accurately process them, which may lead to some false positives.

Compared with dynamic detection methods, static ones do not need to run multi-threaded programs and can mine their possible execution paths, which show better detection precision and efficiency commonly. Hence, we propose a static UAF vulnerability detection method based on a segment Petri net and value flow graph. Our contributions are as follows:

1) We extend the Petri net with segments and accompanying labels, which can model complex control flow causality constraints in a multi-threaded program, e.g., intra-thread, inter-thread, and lock/fork coupling causality constraints;

2) We extend a static value flow graph with segment information and improve its source-sink path analysis method, which can analyze UAF triggering and condition-satisfied constraints caused by different concurrency primitives and complex control structures;

3) We detect UAF vulnerabilities by analyzing compatibility among the above three constraints, which shows better detection recall, precision, and time complexity compared with Canary and Saber on public and synthetic datasets.

This paper is organized as follows. Section II summarizes the related work. Section III introduces the motivation and framework of this work. Section IV formulates some definitions used in UAF vulnerability detection. Section V presents our UAF vulnerability detection method. Section VI demonstrates experimental results. Finally, Section VII concludes this paper.

II. RELATED WORK

A. DYNAMIC DETECTION METHODS

Dynamic detection methods detect UAF vulnerabilities in a multi-threaded program from its running traces, which

can be divided into memory object tracking, aliasing mechanisms, and fuzzing methods. Among them, memory object tracking detects UAF vulnerabilities by tracking each memory object's lifetime. Gao et al. [10] propose a UAF vulnerability detection method based on memory tagging extension for C/C++ programs, which binds memory tags to both allocated objects and their referencing pointers and modifies tags upon release. Their method achieves 98% accuracy on a Juliet test set. Li et al. [11] address the problems of memory objects reoccupation. They track the lifetime of memory objects through runtime libraries, label pointers to memory objects, and track their propagation process. Park et al. [12] address high overhead of delayed memory reuse in UAF vulnerability defense. They consider locality when reusing previously freed heap chunks, adaptively pool the chunks considering the expected lifespan, and are assisted by a tailored operation system service to quickly return physical pages to a system. Experimental results show that their method can reduce 1.5% average overhead in the standard performance evaluation corporation CPU benchmark suite. He et al. [13] detect UAF vulnerabilities based on incorrect reference counting of memory objects. They collect program execution traces by reproducing a UAF vulnerability, identify UAF vulnerability objects and their references, and detect reference miscounting. Their method can diagnose 48 reference counting errors among 76 cases. Ahn et al. [14] manage virtual address layout in user space and use an extended Berkeley packet filter for kernel page handling, which achieves 15% faster than DangZero with 61% lower memory overhead than FFmalloc. Aliasing mechanisms recognize variable aliasing as well as lock use and release operations among program statements to detect UAF vulnerabilities. Gorter et al. [15] improve traditional page protection and aliasing schemes. They use unikernel-like virtualization to access page tables at ring-0 and make released objects inaccessible. Du et al. [16] address the performance overhead problem of key-and-lock mechanisms in existing UAF vulnerability detection methods. They reduce performance overhead by reducing the number of memory accesses related to metadata propagation. Their method maintains key-and-lock's detection rate with 17% average overhead and 8% memory overhead in SPEC CPU 2017. Fuzzing methods use a large number of faulty inputs to effectively detect UAF vulnerabilities. Zhang et al. [17] improve traditional Greybox fuzzing methods. They identify different targets to guide the fuzzing process to cover specific heap operations without wasting resources exploring unrelated program components. Chen et al. [18] discuss whether fuzzing can efficiently identify UAF vulnerabilities with specific patterns and characteristics. They consider the influence of multi-stage triggering, code scope, and non-determinism. Based on this, they provide guidelines to improve UAF vulnerability detection rates. Chen et al. [19] measure the performance of existing UAF fuzzers on 150 real UAF cases (Linux kernel, Python, and Firefox). They categorize the root causes of 150 real UAF cases into

11 patterns, several patterns can be translated into simple static detection rules to cover a large portion of the 150 UAF vulnerabilities with high accuracy.

B. STATIC DETECTION METHODS

Static detection methods analyze source codes of a multi-threaded program, which provide a more comprehensive program behaviour analysis, higher detection precision, and do not require executing it. Existing methods can be divided into value-flow graph-based, source-sink-based, and metric-based methods. Among them, value-flow graph-based methods construct a value-flow graph for a multi-threaded program and detect UAF vulnerabilities by mining value flow relationship in the graph. Saber [20] defines a sparse value-flow graph to track the value flow from allocation to free sites, which can capture defuse chains and value flows via assignments for all memory locations represented by both top-level and address-taken pointers. It detects 254 leaks in the 15 SPEC2000 C programs and seven applications, while keeping the false positive rate at 18.3%. Pinpoint [21] addresses the pointer trap problems of existing sparse value-flow analysis methods. It decomposes the cost of high-precision points-to analysis by precisely discovering local data dependence and delaying the expensive inter-procedural analysis through memorization. An et al. [22] detect UAF vulnerabilities that are caused by dangling pointers. They use a call graph and data flow graph to identify and eliminate potential dangling pointers. Teodorescu and Lucanu [23] design a framework for UAF vulnerability detection in C++ based on an abstract syntax tree and data flow analysis, which can identify dangling pointers and their lifecycles at compile time without source code modification. Source-sink-based methods mine traces of different variables from free to initial in a multi-threaded program and detect UAF vulnerabilities by considering source-sink reachability. Canary [24] transforms UAF vulnerability detection into a source-sink reachability problem. It captures intra-thread and inter-thread constraints by mining data dependence and interference dependence in a value-flow graph, which effectively reduces the interleaving of redundant threads and improves detection efficiency. Zheng et al. [25] propose an abstract syntax tree to represent potential evolution paths that threaten program security. Based on which mutation samples containing vulnerabilities are generated based on paths in the tree. Metric-based methods propose different metrics to measure whether there exist UAF vulnerabilities in a multi-threaded program. Wei et al. [26] analyze potential UAF vulnerability pairs in programs and gather their feature metric information into an evaluation pool. Then, they use controllability metrics to rank the pool, filter out the high ones as candidates, and conduct manual inspection. Bernhard et al. [27] propose a UAF mitigation scheme. They alias multiple virtual pages to one physical page so that they can efficiently embed tag bits into a pointer. In addition, they introduce several optimizations that significantly reduce the performance impact of this approach to memory tagging.

C. SUMMARY

Dynamic detection methods need running traces of a multi-threaded program. Hence, they are difficult to detect all UAF vulnerabilities in a large multi-threaded program because they need its different running traces. Static detection methods analyze source codes of a multi-threaded program. Although static detection methods commonly show good performance, they suffer from causality constraints caused by different concurrency primitives and complex control structures among threads. To address those problems, this work proposes a static UAF vulnerability detection method, which can analyze three constraints based on an improved segment Petri net and value flow graph, based on which UAF vulnerabilities are detected by analyzing compatibility among constraints.

III. MOTIVATION AND FRAMEWORK

In this section, we use a multi-threaded program example to explain what UAF vulnerabilities are and illustrate false positives in Canary, based on which we give our UAF vulnerability detection framework.

A. MULTI-THREADED PROGRAM EXAMPLE

Algorithm 1 shows a multi-threaded program that has a main thread and a child thread. The main thread first allocates memory for pointers v_5 and v_1 , initializes variable v_7 , and changes v_5 's value (lines 2–5). Then, it requests lock ι , starts child thread u , initializes pointer v_3 , outputs v_3 's value, and releases ι (lines 6–10). Next, it requests ι , initializes pointers v_4 , executes a loop structure, and releases ι (lines 11–19). Finally, it waits for u to end (line 20). The child thread first allocates memory for pointer v_2 (line 23). Then, it requests ι , changes pointer v_6 's value, releases v_2 's memory, and releases ι (lines 24–27).

Algorithm 1 has the following two execution situations:

- 1) In the first execution situation, it orderly executes lines 2–10, 11–20, and 23–27;
- 2) In the second execution situation, it orderly executes lines 2–10, lines 23–27, and lines 11–20.

In the first execution situation, there is no UAF vulnerability. In the second execution situation, there is a UAF vulnerability between lines 15 and 26 because pointers v_4 and v_5 point to the same memory while v_5 's memory is released in a child thread.

B. CANARY PERFORMANCE

Considering that existing UAF vulnerability detection methods are mainly based on a static value flow graph, this section gives its formal definition and introduces a classical UAF vulnerability detection tool, i.e., Canary.

Definition 1: A static value flow graph is

$$G = (V, O, R, C, \sigma), \quad (1)$$

where $V = \{n_i = (v_j, l_k) | (i \in \mathbf{N}^+) \wedge (j \in \mathbf{N}^+) \wedge (k \in \mathbf{N}^+)\}$ is a variable node set, n_i is a variable node in G , v_j is a variable, l_k denotes that v_j is located at the k -th line of a multi-

Algorithm 1 Main and Child Threads of a Multi-Threaded Program

```

1: void  $m()$ {
2:   int  $v_5 = (\text{int}^{**})\text{malloc}(\text{sizeof}(\text{int}^{**}));$ 
3:   int  $v_1 = (\text{int}^{*})\text{malloc}(\text{sizeof}(\text{int}));$ 
4:   int  $v_7 = 1;$ 
5:    $*v_5 = v_1;$ 
6:   lock( $\iota$ );
7:   fork( $u, v_5$ );
8:   int  $*v_3 = *v_5;$ 
9:   print( $*v_3$ );
10:  unlock( $\iota$ );
11:  lock( $\iota$ );
12:  int  $*v_4 = *v_5;$ 
13:  for (int  $i = 0; i \leq 1; i++$ ) do
14:    if  $v_7 == 1$  then
15:      print( $*v_4$ );
16:    end if
17:     $v_7 = 2;$ 
18:  end for
19:  unlock( $\iota$ );
20:  join( $u$ );
21: }
22: void  $u(\text{int}^{**} v_6)$ {
23:   int  $*v_2 = (\text{int}^{*})\text{malloc}(\text{sizeof}(\text{int}));$ 
24:   lock( $\iota$ );
25:    $*v_6 = v_2;$ 
26:   free( $v_2$ );
27:   unlock( $\iota$ );
28: }

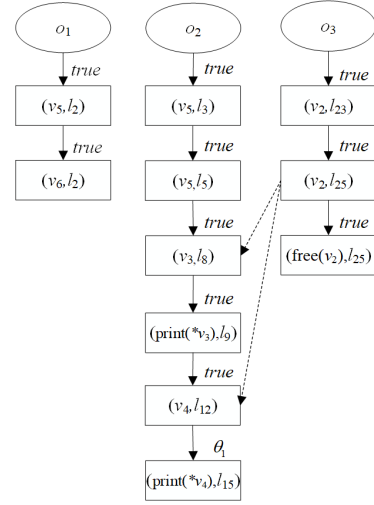
```

threaded program, $\mathbf{N}^+$ is a positive integer set, $O = \{o_y | y \in \mathbf{N}^+\}$ is a memory object node set, $R \subseteq (O \times V) \cup (V \times V)$ is a dependency set that is composed of data dependency and interference dependency, the former represents value flows within a thread and the latter represents value flows among threads, $C = \{c_z | z \in \mathbf{N}^+\}$ is a condition set, and $\sigma : R \rightarrow C$ is a map function between a dependency and its condition.

From Definition 1, we can construct the static value flow graph of Algorithm 1, as shown in Fig. 1. An ellipse node is a memory object node, a rectangle node is a variable node, a solid line is a data dependency, and a dashed line is an interference dependency. A text on a solid or dashed line represents a condition under which a dependency is satisfied.

To detect UAF vulnerabilities in a multi-threaded program from a static value flow graph, Canary extracts source-sink paths in the graph, which are lifecycles of different variables from free to use, analyzes constraints among variables within these paths, and detects UAF vulnerabilities by using SMT [28]. Specifically, Table 1 shows two source-sink paths in Algorithm 1. Then, Canary analyzes the following two constraints:

1) Partial order constraint: It considers causality constraints between two statements within the same thread, between a

**FIGURE 1.** Static value flow graph of Algorithm 1.**TABLE 1.** Source-Sink paths in Algorithm 1.

ID	Source-sink paths
1	$\text{free}(v_2) \rightarrow v_2 \rightarrow v_3 \rightarrow \text{print}(*v_3)$
2	$\text{free}(v_2) \rightarrow v_2 \rightarrow v_4 \rightarrow \text{print}(*v_4)$

fork statement and an initiated thread, and between a join statement and the last operation of an initiated thread. For example, l_5 and l_7 belong to the main thread in Algorithm 1. The former needs to be executed before the latter, which forms a causality constraint and is denoted as $l_5 < l_7$.

2) Consistency constraint: It ensures that the memory object pointed by a direct or indirect value flow process's variable cannot be changed by other concurrent statements when the process is not finished. For example, l_5 , l_8 , and l_{25} form a direct value flow process in Algorithm 1 because l_8 and l_{25} have read and write operations to the same memory, respectively, and l_5 also involves a write operation to the same memory. To ensure v_2 and v_3 point to the same memory after the above direct value flow process, l_5 should be executed before l_{25} and l_{25} should be executed before l_8 , which is denoted as $l_5 < l_{25} < l_8$.

As shown in Table 2, Canary mines four constraints in Algorithm 1. Then, Canary detects two UAF vulnerabilities by using SMT. However, l_{26} and l_9 form a lock/fork coupling causality, l_9 is conducted before l_{26} , and l_8 is conducted before l_{25} . Therefore, the detection result is wrong. The reason is that Canary cannot accurately analyze UAF vulnerabilities in lock/fork coupling situations.

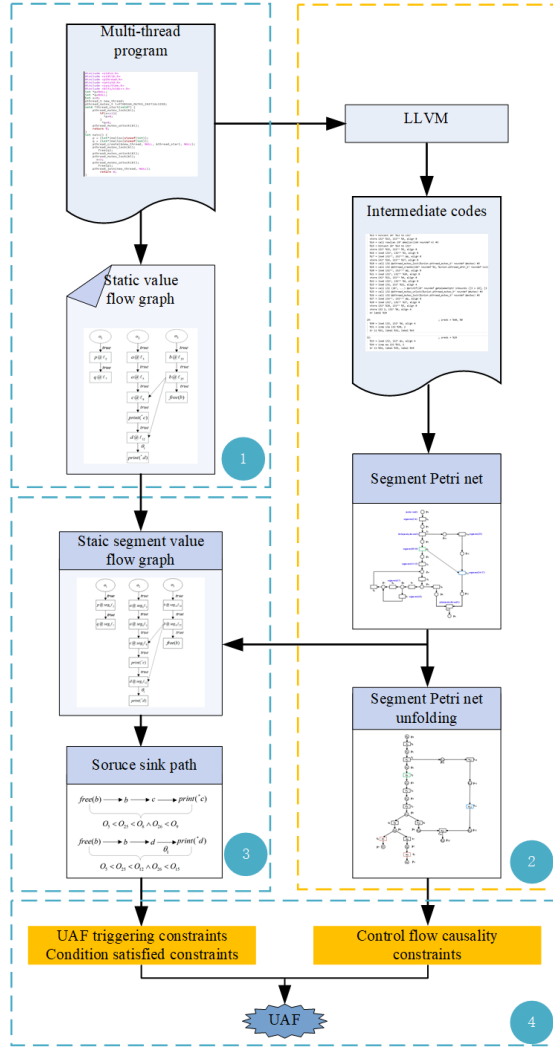
C. FRAMEWORK OF THIS WORK

To solve the drawbacks of existing methods, this work proposes a UAF vulnerability detection method based on a segment Petri net and static segment value flow graph. Its framework is shown in Fig. 2, which has the following four phases:

1) Static value flow graph construction: As shown in phase 1, we analyze source codes of a multi-thread program

TABLE 2. Constraints of Source-Sink paths in Algorithm 1.

ID	Constraints
1	$l_5 < l_8 < l_9 < l_{25} < l_{26}$
2	$l_8 < l_9 < l_{25}$
3	$(l_5 < l_{12} < l_{15} < l_{25} < l_{26} < l_{20}) \vee (l_5 < l_{25} < l_{26} < l_{12} < l_{15} < l_{20})$
4	$l_5 < l_{20}$

**FIGURE 2.** The framework of this work.

and construct its static value flow graph based on Definition 1;

2) Control flow causality constraints analysis: As shown in phase 2, we transform source codes of a multi-thread program into intermediate ones, use a Petri net to model its control flow, and improve a Petri net with segment information to construct a segment Petri net. Then, we analyze control flow causality constraints in a multi-thread program by using a segment Petri net unfolding method, which contain intra-thread, inter-thread, and lock/fork coupling causality constraints;

3) UAF triggering and condition-satisfied constraints analysis: As shown in phase 3, we associate a static value flow graph with segment information in a segment Petri net

to construct a static segment value flow graph. Then, we mine source-sink paths from the graph and analyze UAF triggering and condition-satisfied constraints in them;

4) UAF vulnerability detection: As shown in phase 4, we analyze compatibility among control flow causality, UAF triggering, and condition-satisfied constraints to detect UAF vulnerabilities in a multi-thread program.

Algorithm 2 Intermediate Codes of Algorithm 1.

```

1: malloc(m, v5);
2: malloc(m, v1);
3: write(m, v7);
4: read(m, v1);
5: write(m, v5);
6: acq(m, l);
7: fork(m, u);
8: read(m, v5);
9: write(m, v3);
10: use(m, v3);
11: rel(m, l);
12: acq(m, l);
13: read(m, v5);
14: write(m, v4);
15: branch(i ≤ 1) l16, l23;
16: label l16;
17: read(m, v7);
18: branch(v7 == 1) l19, l21;
19: label l19;
20: use(m, v4);
21: label l21;
22: write(m, v7);
23: label l23;
24: rel(m, l);
25: join(m, u);
26: malloc(u, v2);
27: acq(u, l);
28: read(u, v2);
29: write(u, v6);
30: free(u, v2);
31: rel(u, l);

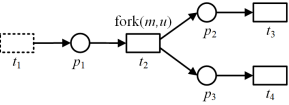
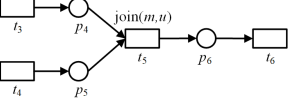
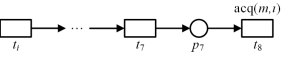
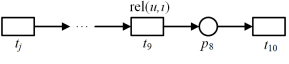
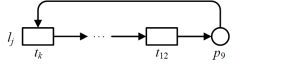
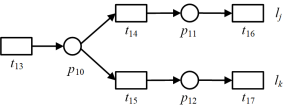
```

IV. FORMAL SPECIFICATION

A. INTERMEDIATE CODE REPRESENTATION

To balance expressive performance and complexity of a segment Petri net, we transform source codes of a multi-threaded program into intermediate ones that follow LLVM standards [29]. The intermediate codes of Algorithm 1 are

TABLE 3. Construction methods of a segmented Petri net.

Program statements	Petri net fragment	Description
Initial Petri net		We use a place p_0 and its successor transition t_0 to initialize a Petri net. The number of p_0 's tokens is 1 and t_0 's associated statement list is empty.
fork(m, u)		We use t_1 to represent statements before fork(m, u) in a thread m , add a place p_1 and transition t_2 as t_1 's successors, and associate statement fork(m, u) with t_2 . Two place and transition pairs are added as t_2 's successors, i.e., p_2 and t_3 and p_3 and t_4 . The former is used to describe control flow after fork(m, u) in m . The latter is used to describe control flow in thread u .
join(m, u)		We add places p_4 and p_5 after t_3 and t_4 , respectively, add a transition t_5 as p_4 's and p_5 's successor, and associate a statement join(m, u) with t_5 . Then, we add a place p_6 and transition t_6 as t_5 's successors to indicate control flow after join(m, u) in m .
acq(m, ι)		We use t_7 to represent statements before fork(m, u) in m , add a place p_7 and transition t_8 as t_7 's successors, and associate statement acq(m, ι) with t_8 . Then, we find a transition t_i that satisfies the following three conditions: t_i 's thread is not m , t_i has a lock request statement, and there is no lock release statement between t_i and t_7 . If t_i exists, we mark t_7 with a blue color and a lock/fork/acq label.
rel(m, ι)		We use t_9 to represent statement rel(u, ι) in m , add a place p_8 and transition t_{10} as t_9 's successors, and set t_{10} 's associated statement list to empty. Then, we find a transition t_j with an acq operation before t_9 . If there exist transitions with fork operations between t_j and t_9 , we mark t_{10} with a green color and a lock/fork/rel label.
label l_j		We associate a transition t_{11} with a mark l_j .
branch l_j		We add a place p_9 after a transition t_{12} and set a transition t_k with a mark l_j as p_9 's successor.
branch(c_z) l_j, l_k		We add a place p_{10} as t_{13} 's successor and add two transition and place pairs as p_{10} 's successors, i.e., t_{14} and p_{11} and t_{15} and p_{12} . Then, we set a transition t_{16} with a mark l_j as p_{11} 's successor and set a transition t_{17} with a mark l_k as p_{12} 's successor.
statement s		If a statement does not add any place, transition, or control flow to a thread, statement s is added to the current transition's associated statement list.

shown in Algorithm 2 and their symbol meanings are as follows:

- 1) malloc(m, v_i): It means that thread m allocates memory space for variable v_i ;
- 2) use(m, v_i): It means that m uses v_i ;
- 3) free(m, v_i): It means that m releases v_i 's memory space;
- 4) read(m, v_i): It means that m reads v_i 's value;
- 5) write(m, v_i): It means that m updates v_i 's value;
- 6) acq(m, ι): It means that m acquires lock ι ;
- 7) rel(m, ι): It means that m releases ι ;
- 8) fork(m, u): It means that m activates thread u ;
- 9) join(m, u): It means that m waits for u to terminate;
- 10) label l_j : It means that a sign is associated with the j -th line of a multi-threaded program's intermediate codes;
- 11) branch l_j : It means that a thread jumps to label l_j without conditions;
- 12) branch(c_z) l_j, l_k : It means that a thread jumps to label l_j if condition c_z is satisfied. Otherwise, it jumps to label l_k .

B. CONSTRAINT DEFINITION

To address incomplete constraint problems of existing methods, this work refines three constraints among statements,

i.e., control flow causality, UAF triggering, and condition-satisfied constraints.

Definition 2: For an execution statement sequence $\Gamma = (l_i | i \in \mathbb{N}^+)$ of a multi-threaded program, where l_i is the i -th line in the program, *control flow causality constraints* have the following three types:

- 1) Assuming that $l_j \in \Gamma$ and $l_k \in \Gamma$ are in the same thread and $j < k$, there exists an intra-thread causality between them;
- 2) Assuming that $l_j \in \Gamma$ is a fork operation in start thread u , $i < k$, and $l_k \in \Gamma$ is a statement in u , there exists a inter-thread fork causality between $l_i \in \Gamma$ and l_k . Assuming that a statement $l_j \in \Gamma$ is a join operation to wait thread u , $j < i$, and $l_k \in \Gamma$ is a statement in u , there exists a inter-thread join causality between l_k and $l_i \in \Gamma$;

3) Assuming that $l_i \in \Gamma$ and $l_k \in \Gamma$ are lock request and release operations, respectively, $i < j < k$, statement $l_j \in \Gamma$ is a fork operation to start thread u , $y \in (j, k)$, and a statement $l_z \in \Gamma$ is in an initialized thread, there exists a lock/fork coupling causality between $l_y \in \Gamma$ and l_z .

Definition 3: For an execution statement sequence $\Gamma = (l_i | i \in \mathbb{N}^+)$ in a multi-threaded program, *UAF triggering constraints* have the following two types:

1) Assuming that statements $l_j \in \Gamma$ and $l_k \in \Gamma$ are write and read operations to the same memory, respectively, and l_k needs to read the memory wrote by l_j , if there exists another memory write operation in statement $l_i \in \Gamma$, it should be conducted before l_j , which is a read/write causality between them;

2) Assuming that statements $l_j \in \Gamma$ and $l_k \in \Gamma$ are variable use and free operations, respectively. If l_k is conducted before l_j , there exists a UAF triggering constraint.

Definition 4: For an execution statement sequence $\Gamma = (l_i | i \in \mathbb{N}^+)$ in a multi-threaded program, and $\theta_1, \theta_2, \theta_3, \dots, \theta_j$ are dependency conditions among statements, a *condition-satisfied constraint* is a conjunction operation among these conditions, which is denoted as $\theta_1 \wedge \theta_2 \wedge \theta_3 \wedge \dots \wedge \theta_j$.

From Definitions 2-4, control flow causality constraints are constraints that a multi-threaded program must follow and this work mines them from a segmented Petri net to be introduced; UAF triggering constraints are constraints that can trigger UAF vulnerabilities and condition-satisfied constraints consider conditions among statements. They are mined from a static segmented value flow graph to be introduced.

C. SEGMENTED PETRI NET

Existing methods [30] construct a segment graph for a multi-threaded program based on its execution traces and analyze UAF vulnerabilities. They suffer from inaccurate constraint analysis and high complexity when there exist loop or branch structures. Hence, this work transforms source codes of a multi-threaded program into intermediate ones, improves a Petri net into a segmented one to model concurrent behavior of a multi-threaded program, and analyzes the above constraints from a segmented Petri net.

Definition 5: A segmented Petri net is

$$\Sigma = (P, T, F, S, M_0, \lambda), \quad (2)$$

where $P = \{p_1, p_2, \dots, p_e\}$ is a place set, e is the number of places, $p_i \in P$ is a place, $T = \{t_1, t_2, \dots, t_h\}$ is a transition set, h is the number of transitions, $t_j \in T$ is a transition, $F \subseteq (P \times T) \cup (T \times P)$ is a flow relationship set, $S = \{s_1, s_2, \dots, s_w\}$ is a segment set, w is the number of segments, $M_0 : P \rightarrow \{0, 1, 2, \dots\}$ is Σ 's initial token setting, and $\lambda : (P \cup T) \rightarrow S$ is a segment mapping function.

Table 3 shows construction methods of a segmented Petri net from intermediate codes of a multi-threaded program. Taking Algorithm 1 as an example, their segmented Petri net is shown in Fig. 3. At the initial state, t_0 can be executed only. Then, p_1 has a token and t_1 can be executed. Next, p_2 and p_{v0} have a token, the transitions after p_2 belong to a main child, and the transitions after p_{v0} belong to a child thread. To characterize lock/fork coupling causality constraints, this work deals with changes caused by executing lock requests and releases and designs two accompanying labels $\alpha(m, \iota, u)$ and $\beta(m, \iota, u)$. The former indicates that thread m requests lock ι when thread u holds ι . The latter indicates that thread m starts thread u while holding ι and then releases ι . There is a

lock/fork coupling causality constraint between them, which is represented by a dotted line in a segmented Petri net.

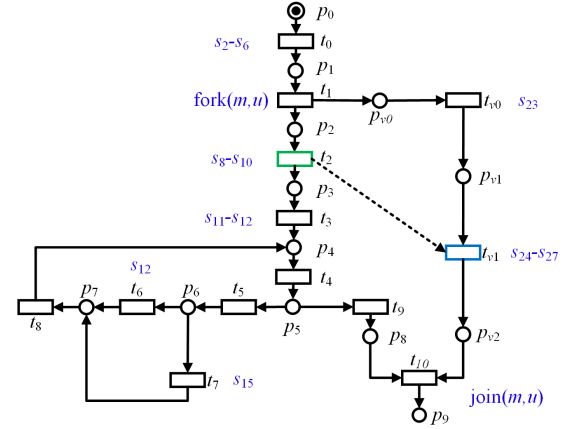


FIGURE 3. Segmented Petri net of Algorithm 1.

D. STATIC SEGMENTED VALUE FLOW GRAPH

To realize information consistent between a static value flow graph and segment Petri net and accurately mine constraints in a multi-threaded program, we associate segment information with a static value flow graph to construct a static segment value flow graph.

Definition 6: A static segment value flow graph is

$$\wp = (O, \hat{V}, R, C, S, \sigma), \quad (3)$$

where $\wp$ is a static segment value flow graph, $\hat{V} = \{n_i = (v_j, s_z, l_k) | ((i \in \mathbb{N}^+) \wedge (s_z \in S) \wedge (k \in \mathbb{N}^+))\}$ is variable node set, and s_z is a segment that is drawn from the segment set S of corresponding segmented Petri net Σ .

The construction method of a static segment value flow graph is shown in Algorithm 3, which inputs segmented Petri net Σ and static value flow graph G and outputs static segment value flow graph $\wp$. First, we initialize an empty static segment value flow graph $\wp$, set $\wp$'s memory object node set, dependency set, condition set, and map function as the same as those in G , and set $\wp$'s segment set as the same as that in Σ (lines 2–4). Then, for each variable node in G , we find its corresponding segment in Σ , supplement each variable node with segmented information, and add it into $\wp$ (lines 5–8). The static segmented value flow graph of Algorithm 1 is shown in Fig. 4.

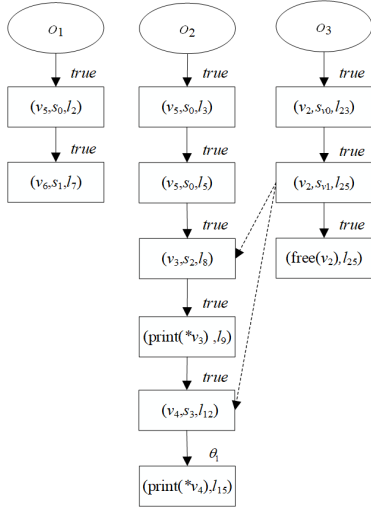
V. UAF VULNERABILITIES DETECTION METHOD

A. SOURCE-SINK PATH ANALYSIS

Algorithm 4 introduces how to mine source-sink paths of different variables from a static segment value flow graph, which inputs static segment value flow graph $\wp$ and outputs source-sink paths. First, it initializes an empty source-sink path set Λ (line 2). Then, it traverses variable nodes and finds those with free statements in $\wp$ (lines 3–4). Next, it initializes a source-sink path ρ , adds a variable node with free statements into ρ , and finds other variable nodes with

Algorithm 3 Construct a Static Segment Value Flow Graph

- 1: **Input:** Segmented Petri net $\Sigma = (P, T, F, S, M_0, \lambda)$ and static value flow graph $G = (O, V, R, C, \sigma)$.
- 2: Initialize a static segment value flow graph $\wp$;
- 3: Set $\wp$'s memory object node set O , dependency set R , condition set C , and σ as the same as those in G ;
- 4: Set $\wp$'s segment set S as the same as that in G ;
- 5: **for** $(v_j, l_k) \in V$ **do**
- 6: Find l_k 's segment s_z in Σ ;
- 7: Add (v_j, s_z, l_k) into $\wp$'s $\tilde{V}$;
- 8: **end for**
- 9: **Output:** $\wp$.

**FIGURE 4.** Static segment value flow graph of Algorithm 1.

use statements to the same memory object (lines 5–8). Then, it measures reachability between a free statement and a used one. If they are reachable, it adds a node into ρ and calculates UAF triggering and condition-satisfied constraints between them (lines 9–14). Finally, it outputs Λ (line 18).

B. SEGMENT PETRI NETS UNFOLDING

Although a source-sink path tracks a trace of a variable from free to use, we need to analyze control flow causality constraints to avoid false positives. For example, if two statements are a lock request and release, we need to determine whether they can be concurrent. Hence, we associate a statement in constraints with a transition in a segment Petri net and determine concurrency and conflict events among transitions based on segment Petri net unfolding by using an occurrence net.

An occurrence net can describe transition firing and token movement in a segment Petri net, which can be used to analyze its concurrent and conflict events. Specifically, a node in an occurrence net is defined as an event or a condition. The former denotes a transition execution and the latter denotes a token state in a segment Petri net. An edge in an occurrence net represents a token generating

Algorithm 4 Source-Sink Path Analysis

- 1: **Input:** Static segment value flow graph $\wp$.
- 2: Initialize a source-sink path set $\Lambda = \{\}$;
- 3: **for** $n_i = (v_j, s_z, l_k) \in \hat{V}$ **do**
- 4: **if** l_k is a free statement **then**
- 5: Initialize $\rho = \{\}$;
- 6: $\rho = \rho \cup n_i$;
- 7: Assuming that v_j 's memory object is o_m ;
- 8: Constructing variable node set $\tilde{V}$ that has operations with o_m ;
- 9: **for** Variable node $n_y \in \tilde{V}$ **do**
- 10: **if** n_i and n_y is reachable **then**
- 11: $\rho = \rho \cup n_y$;
- 12: Calculate UAF triggering and condition-satisfied constraints between n_i and n_y ;
- 13: **end if**
- 14: **end for**
- 15: $\Lambda = \Lambda \cup \rho$;
- 16: **end if**
- 17: **end for**
- 18: **Output:** Λ .

or consuming during transition execution in a segment Petri net. A configuration represents a possible execution sequence in a segment Petri net and a local configuration represents two adjacent events. If a local configuration appears more than once, its start event is defined as a cut-off event and represented as a red box in an occurrence net. The occurrence net of the segment Petri net in Fig. 3 is shown in Fig. 5.

C. LOOP STRUCTURE ANALYSIS

Because a static segment value flow graph cannot derive a specific number of expansions of a loop structure due to a lack of runtime information, existing methods set a fixed number of expansions to process it through CBMC [31]. However, they are difficult to achieve a balance between efficiency and accuracy. Considering that a cut-off event in an occurrence net describes a state that is the same as before, e.g., the first and second executions of a loop structure, we introduce a loop structure processing method, i.e., we find a cut-off event's source event and use all events between them to construct an expansion segment for a segment Petri net. This segment corresponds to a loop structure in a multi-threaded program, which is called a Petri net expansion segment accompanying a cut-off event. The accompanying Petri net expansion segment corresponding to cut-off event e_9 in Fig. 5 is shown in Fig. 6.

D. UAF VULNERABILITY DETECTION

In Section V-A, we mine source-sink paths from a static segment value flow graph and analyze their UAF triggering and condition-satisfied constraints. In Section V-B, we analyze control flow causality constraints based on segment Petri nets unfolding. In this section, we introduce a UAF vulnerability detection method based on loop structure and compatibility

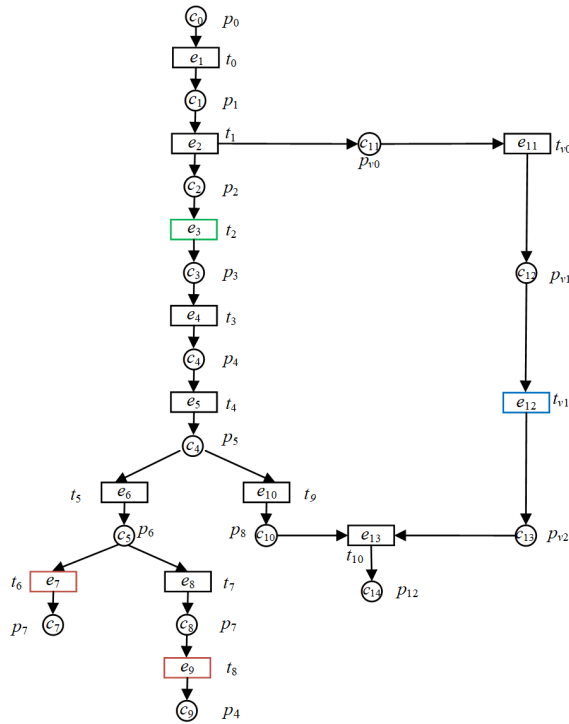


FIGURE 5. Occurrence net of a segment Petri net.

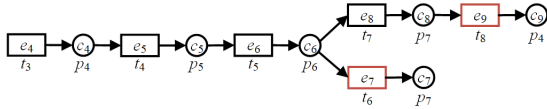


FIGURE 6. Petri net expansion segment accompanying a cut-off event.

analysis among control flow causality, UAF triggering, and condition-satisfied constraints.

The proposed UAF vulnerability detection method is shown in Algorithm 5, which inputs a source-sink path set Λ and outputs source-sink paths that have UAF vulnerabilities. First, Algorithm 5 initializes an empty source-sink path set $\hat{\Lambda}$ (line 2). Then, for each source-sink path in Λ , Algorithm 5 gets its control flow causality, UAF triggering, and condition-satisfied constraints (line 4). If a source-sink path's condition-satisfied constraints are not satisfied, Algorithm 5 checks the next source-sink path. Otherwise, it checks compatibility of control flow causality and UAF triggering constraints (lines 5–42). Specifically, for each constraint in a UAF triggering constraint, Algorithm 5 finds its corresponding transitions t_1 and t_2 (lines 5–7). If they are in the same Petri net expansion segment accompanying a cut-off event, their control flow causality constraints are compatible (lines 8–9). If they have a lock/fork coupling causality and $t_1 < t_2$, their control flow causality constraints are compatible (lines 10–13). Otherwise, their control flow causality constraints are not compatible and this source-sink path is added into $\hat{\Lambda}$ (lines 14–18). If they are different and concurrent, there is no sequential relationship between them (lines 19–20). If they are different and concurrent

Algorithm 5 UAF Vulnerabilities Detection Method

```

1: Input: Source-sink path set  $\Lambda$ .
2:  $\hat{\Lambda} = \{\}$ ;
3: for  $\rho \in \Lambda$  do
4:   Assuming that  $\rho$ 's control flow causality, UAF triggering, and condition-satisfied constraints are  $\varsigma$ ,  $\zeta$ , and  $\xi$ , respectively;
5:   if constraints in  $\xi$  are satisfied; then
6:     for constraint  $n_i < n_j$  in  $\zeta$  do
7:       Assuming that  $n_i$ 's corresponding transition and  $n_j$ 's are  $t_1$  and  $t_2$ , respectively;
8:       if  $t_1$  and  $t_2$  are in the same Petri net expansion segment accompanying a cut-off event then
9:         continue;
10:      else if  $t_1$  and  $t_2$  is lock/fork coupling causality then
11:        if  $t_1 < t_2$  then
12:           $n_i$  and  $n_j$  and  $\varsigma$  are compatible;
13:          continue;
14:        else
15:           $n_i$  and  $n_j$  and  $\varsigma$  are not compatible;
16:           $\hat{\Lambda} = \hat{\Lambda} \cup \rho$ 
17:          break;
18:        end if
19:      else if  $t_1 \neq t_2$  and they are concurrent then
20:        continue;
21:      else if  $t_1 \neq t_2$  and they are not concurrent then
22:        if  $t_1 < t_2$  then
23:           $n_i$  and  $n_j$  and  $\varsigma$  are compatible;
24:          continue;
25:        else
26:           $n_i$  and  $n_j$  and  $\varsigma$  are not compatible;
27:           $\hat{\Lambda} = \hat{\Lambda} \cup \rho$ 
28:          break;
29:        end if
30:      else
31:        Assuming that  $n_i$ 's line number and  $n_j$ 's are  $l_y$  and  $l_z$ , respectively;
32:        if  $l_y > l_z$  then
33:           $n_i$  and  $n_j$  and  $\varsigma$  are compatible;
34:          continue;
35:        else
36:           $n_i$  and  $n_j$  and  $\varsigma$  are not compatible;
37:           $\hat{\Lambda} = \hat{\Lambda} \cup \rho$ 
38:          break;
39:        end if
40:      end if
41:    end for
42:  end if
43: end for
44: Output:  $\hat{\Lambda}$ .

```

and $t_1 < t_2$, their control flow causal transitions are compatible (lines 21–24). Otherwise, their control flow

TABLE 4. UAF vulnerability detection results of Canary and our method.

Program	Thread counts	UAF vulnerability counts	Detected UAF vulnerability counts		Recall		Precision	
			Canary	Our Method	Canary	Our Method	Canary	Our Method
CWE416-1	2	1	0	1	0%	100%	0%	100%
CWE416-2	2	1	1	1	100%	100%	100%	100%
CWE416-3	2	1	1	1	100%	100%	100%	100%
CWE416-4	2	1	1	1	100%	100%	100%	100%
CWE416-5	2	1	2	1	100%	100%	50%	100%
Test1	2	2	1	2	50%	100%	50%	100%
Test2	2	1	0	1	0%	100%	0%	100%
Test3	2	1	2	1	100%	100%	50%	100%
Test4	3	2	3	2	100%	100%	66.7%	100%
Test5	2	1	1	2	100%	100%	100%	50%

TABLE 5. UAF vulnerability detection results of Saber and our method.

Program	UAF vulnerability counts	Detected UAF vulnerability counts		Recall		Precision	
		Saber	Our Method	Saber	Our Method	Saber	Our Method
CWE416	393	298	347	26.7%	59.2%	35.2%	67.1%
CWE416-1	1	1	1	100%	100%	100%	100%
CWE416-2	1	1	1	100%	100%	100%	100%
CWE416-3	1	1	1	100%	100%	100%	100%
CWE416-4	1	1	1	100%	100%	100%	100%
CWE416-5	1	2	1	100%	100%	50%	100%
Test1	2	2	2	100%	100%	100%	100%
Test2	1	1	1	100%	100%	100%	100%
Test3	1	2	1	100%	100%	50%	100%
Test4	2	3	2	100%	100%	66.7%	100%
Test5	1	2	2	100%	100%	50%	50%

causality constraints are not compatible and this source-sink path is added into $\hat{\Lambda}$ (lines 25–29). Assuming that l_y and l_z are line numbers of a constraint's corresponding nodes, if $l_y > l_z$, its control flow causality constraints are compatible (lines 31–34). Otherwise, control flow causality constraints are not compatible and this source-sink path is added into $\hat{\Lambda}$ (lines 35–39). Finally, Algorithm 5 outputs $\hat{\Lambda}$ (line 44).

VI. EXPERIMENTAL RESULTS AND DISCUSSION

A. EXPERIMENT SETTINGS

To verify the effectiveness of our method, we compare our method with Canary [24] and Saber [20] on public and synthetic multi-threaded program datasets, which is conducted on a computer with Intel(R) Core(TM) i9-13900HX CPU, 60 GB memory, and Ubuntu 16.2.4 operating system. The details of two datasets are as follows:

- 1) Public dataset: We use several C/C++ multi-threaded programs with UAF vulnerabilities in Juliet Test Suite [32] as public dataset, which contains 393 UAF vulnerabilities.
- 2) Synthetic dataset: We construct five multi-threaded programs with UAF vulnerabilities as a synthetic dataset.

B. EXPERIMENTAL RESULTS

1) UAF VULNERABILITY DETECTION PERFORMANCE

The performance of Canary and our method on public and synthetic datasets in terms of detected UAF vulnerability counts, recall, and precision is shown in Table 4. Canary's detected UAF vulnerability counts are less than our method's,

e.g., CEW416-1, Test1, Test2, and Test5. This is because Canary cannot detect UAF vulnerabilities when a free statement and a use one in the same thread. In other programs, although the detected UAF vulnerability counts of Canary are larger than our method, it has several false positives, e.g., CEW416-5 and Test4. This is because Canary only considers causality among threads and cannot identify lock/fork coupling causality. In addition, our method shows higher recall and precision than Canary, which indicates that our method shows higher UAF vulnerability capture ability and detection reliability. This is because this work can mine more constraints from intermediate codes of multi-threaded programs than Canary. Hence, our method shows better performance than Canary on public and synthetic datasets in terms of detected UAF vulnerability counts, recall, and precision.

The performance of Saber and our method on public and synthetic datasets in terms of detected UAF vulnerability counts, recall, and precision is shown in Table 5. Our method can detect more UAF vulnerabilities than Saber because our method refines more constraints among statements than Saber, e.g., intra-thread causality, inter-thread join causality, lock/fork coupling causality, read/write causality, UAF triggering, and condition-satisfied constraints. In addition, our method's recall and precision are higher than Saber's, which indicates that our method shows higher UAF vulnerability capture ability and detection reliability. This is because Saber only considers whether it is reachable from a source point

TABLE 6. Time complexity performance of Canary, Saber, and our method.

Program	Thread counts	UAF vulnerability counts	Time complexity		
			Canary	Saber	Our Method
CWE416-1	2	1	33 ms	42 ms	15 ms
CWE416-2	2	1	38 ms	50 ms	17 ms
CWE416-3	2	1	32 ms	40 ms	14 ms
CWE416-4	2	1	35 ms	47 ms	15 ms
CWE416-5	2	1	32 ms	39 ms	13 ms
Test1	2	2	12 ms	21 ms	5 ms
Test2	2	1	10 ms	18 ms	4 ms
Test3	2	1	8 ms	15 ms	4 ms
Test4	3	2	9 ms	16 ms	3 ms
Test5	2	1	11 ms	19 ms	4 ms

to a sink one in a value flow graph and cannot identify lock/fork coupling causality. Hence, our method shows better performance than Saber on public and synthetic datasets in terms of detected UAF vulnerability counts, recall, and precision.

In summary, our methods show better performance on public and synthetic datasets than Canary and Saber in terms of recall and precision. Specifically, the recall rate and precision of our method are 25% and 33.3% higher than those of Canary, respectively. The recall rate and precision of our method are 3% and 15.1% higher than those of Saber, respectively.

2) TIME COMPLEXITY PERFORMANCE

The time complexity performance of Canary, Saber, and our method is shown in Table 6. Our method's time complexity is lower than Canary's and Saber's. This is because our method analyzes compatibility among control flow causality, UAF triggering, and condition-satisfied constraints, i.e., there is no UAF vulnerability in a source-sink path if they are incompatible.

Overall, our method shows better performance on public and synthetic datasets than Canary and Saber in terms of time complexity. Specifically, the time complexity of our method is reduced by 12.6 ms and 21.3 ms compared with Canary and Saber, respectively. Hence, our method has better UAF vulnerability capture ability and detection reliability as well as lower calculation resource consumption than the existing two methods.

VII. CONCLUSION

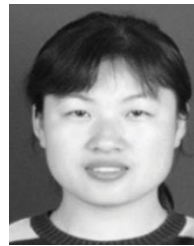
This work focuses on UAF vulnerability detection for multi-threaded programs. It improves a segment Petri net and static segment value flow graph. The former is used to model control flow causality constraints and the latter is used to model UAF triggering and condition-satisfied constraints, based on which this work detects UAF vulnerabilities by analyzing compatibility among these constraints. Experimental results show that our method shows better performance on two datasets than Canary and Saber in terms of detection recall, precision, and time complexity. In the future, we will study dynamic-static-combining methods to improve UAF

vulnerability detection precision in while, do-while, and switch structures. In addition, the proposed control flow causality constraints can be used to detect deadlocks, data races, and memory leaks, which we will study. Finally, we will study how to repair detected UAF vulnerabilities in multi-threaded programs.

REFERENCES

- [1] T. Okamoto, "Prevention of DoS attacks on use-after-free vulnerabilities in mosquito," *Proc. Comput. Sci.*, vol. 207, pp. 1763–1772, Jan. 2022.
- [2] S. Wu, Z. Wang, Z. Yu, X. Li, J. Lin, and Z. Chen, "Application prospect of deadlock detection and removal technology in modern operating system," *Academic J. Eng. Technol. Sci.*, vol. 7, no. 3, pp. 48–53, 2024.
- [3] W. Duo, X. Jiang, O. Karoui, X. Guo, D. You, S. Wang, and Y. Ruan, "A deadlock prevention policy for a class of multithreaded software," *IEEE Access*, vol. 8, pp. 16676–16688, 2020.
- [4] Y. Zhang, H. Liu, and L. Qiao, "Context-sensitive data race detection for concurrent programs," *IEEE Access*, vol. 9, pp. 20861–20867, 2021.
- [5] L. Bo, S. Jiang, J. Qian, R. Wang, and Y. Yao, "Performance evaluation of data race detection based on thread sharing analysis with different granularities: An empirical study," *IEEE Access*, vol. 7, pp. 73819–73829, 2019.
- [6] S. Cai, K. Chen, M. Liu, X. Liu, Y. Wu, and W. Zheng, "Garbage collection and data recovery for N2DB," *Tsinghua Sci. Technol.*, vol. 27, no. 3, pp. 630–641, Jun. 2022.
- [7] C. Vulnerabilities. (2005). *Common Vulnerabilities and Exposures*. [Online]. Available: <https://www.cve.org/About/Metrics>
- [8] C. Liu, Q. Zeng, H. Duan, L. Wang, J. Tan, C. Ren, and W. Yu, "Petri net based data-flow error detection and correction strategy for business processes," *IEEE Access*, vol. 8, pp. 43265–43276, 2020.
- [9] F. Lu, F. Lv, M. Cui, Y. Bao, and Q. Zeng, "Petri net unfolding-based detection and replay of program deadlocks," *IEEE Access*, vol. 12, pp. 53713–53738, 2024.
- [10] L. Gao, C. Hu, and Y. Zhuang, "UAF vulnerability detection method based on dynamic tag extension," in *Proc. IEEE 6th Int. Seminar Artif. Intell., Netw. Inf. Technol. (AINIT)*, Shenzhen, China, Apr. 2025, pp. 991–995.
- [11] R. Li, X. Wang, B. Zhang, and C. Tang, "A memory object sensitive detecting method for use-after-free vulnerabilities," in *Proc. 2nd Int. Conf. Artif. Intell., Hum.-Comput. Interact. Robot. (AIHCIR)*, Tianjin, China, Dec. 2023, pp. 543–547.
- [12] C. Park, Y. Kwak, and H. Moon, "Efficient use-after-free prevention with pooling, OS-assisted, and opportunistic page-level sweeping," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 4768–4782, Sep. 2025, doi: [10.1109/TDSC.2025.3553516](https://doi.org/10.1109/TDSC.2025.3553516).
- [13] L. He, H. Hu, P. Su, Y. Cai, and Z. Liang, "FreeWill: Automatically diagnosing use-after-free bugs by reference miscounting detection on binaries," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2497–2512.
- [14] J. Ahn, J. Lee, K. Lee, W. Gwak, M. Hwang, and Y. Kwon, "BUDAlloc: Defeating use-after-free bugs by decoupling virtual address management from kernel," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 181–197.
- [15] F. Gorter, K. Koning, H. Bos, and C. Giuffrida, "DangZero: Efficient use-after-free detection via direct page table access," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 1307–1322.

- [16] Y. Du, Y. Guo, Y. Zhang, and J. Yang, "RTT-UAF: Reuse time tracking for use-after-free detection," in *Proc. 38th ACM Int. Conf. Supercomputing*, Kyoto, Japan, May 2024, pp. 376–387.
- [17] Y. Zhang, Z. Wang, W. Yu, and B. Fang, "Multi-level directed fuzzing for detecting use-after-free vulnerabilities," in *Proc. IEEE 20th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Shenyang, China, Oct. 2021, pp. 569–576.
- [18] Z. Chen, J. Xiao, A. Stavrou, and H. Wang, "An exploration of fuzzing for discovering use-after-free vulnerabilities," in *Proc. IEEE 35th Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Tsukuba, Japan, Oct. 2024, pp. 19–24.
- [19] Z. Chen, D. Liu, J. Xiao, and H. Wang, "All use-after-free vulnerabilities are not created equal: An empirical study on their characteristics and detectability," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 623–638.
- [20] Y. Sui, D. Ye, and J. Xue, "Static memory leak detection using full-sparse value-flow analysis," in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2012, pp. 254–264.
- [21] Q. Shi, X. Xiao, R. Wu, J. Zhou, G. Fan, and C. Zhang, "Pinpoint: Fast and precise sparse value flow analysis for million lines of code," in *Proc. 39th ACM SIGPLAN Conf. Program. Lang. Design Implement.*, Jun. 2018, pp. 693–706.
- [22] X. An, Q. Zhou, H. Du, Z. Song, and X. Jia, "Refining use-after-free defense: Eliminating dangling pointers in registers and memory," in *Proc. 30th Asia-Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2023, pp. 493–502.
- [23] V.-A. Teodorescu and D. Lucanu, "Static analysis framework for detecting use-after-free bugs in C++," in *Proc. EPTCS*, vol. 410, 2024, pp. 1–7.
- [24] Y. Cai, P. Yao, and C. Zhang, "Canary: Practical static detection of inter-thread value-flow bugs," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Design Implement.*, Jun. 2021, pp. 1126–1140.
- [25] W. Zheng, P. Deng, K. Gui, and X. Wu, "An abstract syntax tree based static fuzzing mutation for vulnerability evolution analysis," *Inf. Softw. Technol.*, vol. 158, Jun. 2023, Art. no. 107194.
- [26] H. Wei, L. Chen, X. Nie, Z. Zhang, Y. Zhang, and G. Shi, "An efficient metric-based approach for static use-after-free detection," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCLOUD/SocialCom/SustainCom)*, Dec. 2022, pp. 58–65.
- [27] L. Bernhard, M. Rodler, T. Holz, and L. Davit, "XTag: Mitigating use-after-free vulnerabilities via software-based pointer tagging on Intel x86-64," in *Proc. IEEE 7th Eur. Symp. Secur. Privacy (EuroS&P)*, Genoa, Italy, Jun. 2022, pp. 502–519.
- [28] L. D. Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Int. Conf. Tools Algorithms Construct. Anal. Syst.*, 2008, pp. 337–340.
- [29] J. Zhao, S. Nagarakatte, M. M. K. Martin, and S. Zdancewic, "Formalizing the LLVM intermediate representation for verified program transformations," in *Proc. 39th Annu. ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, Philadelphia, United States, Jan. 2012, pp. 427–440.
- [30] F. Lu, J. Zheng, Y. Bao, Q. Zeng, H. Duan, and X. Wang, "Deadlock detection of multithreaded programs based on lock-augmented segmentation graph," *J. Softw.*, vol. 32, no. 6, pp. 1682–1700, 2021.
- [31] D. Kroening and M. Tautschnig, "CBMC—C bounded model checker," in *Proc. 20th Int. Conf. Tools Algorithms Construct. Anal. Syst.*, 2014, pp. 389–391.
- [32] T. Boland and P. E. Black, "Juliet 1.1 C/C++ and Java test suite," *Computer*, vol. 45, no. 10, pp. 88–90, Oct. 2012.



YUNXIA BAO received the M.S. degree in application mathematics from Shandong University of Science and Technology, Qingdao, China, in 2005. She is currently an Associate Professor with the College of Mathematics and System Science, Shandong University of Science and Technology. Her research interests include statistics and petri nets.



FAMING LU (Member, IEEE) received the Ph.D. degree in computer software and theory from Shandong University of Science and Technology, Qingdao, China, in 2013. He is currently a Professor with Shandong University of Science and Technology. He has published more than 30 articles in academic journals. His research interests include reinforcement learning and intelligent traffic systems.



CHENG YU received the B.S. degree in computer science and technology and the M.S. degree in computer technology from Shandong University of Science and Technology, Qingdao, China, in 2022 and 2025, respectively. He is currently with Qingdao Municipal Bureau of Statistics. His research interests include software formal verification.



CONG LIU received the B.S. and M.S. degrees in computer software and theory from Shandong University of Science and Technology, Qingdao, China, in 2013 and 2015 respectively, and the Ph.D. degree from the Department of Mathematics and Computer Science, Eindhoven University of Technology. He is an Invited Full Professor with the NOVA Information Management School, Nova University of Lisbon. His research interests include process mining, business process management, and artificial intelligence.



SHANSHAN LI received the B.S. degree in information management and information systems and the M.S. degree in computer technology from Shandong University of Science and Technology, Qingdao, China, in 2015 and 2018, respectively. Her research interests include program verification, testing, and repair.